Pilha Dinâmica

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

- Uma das estruturas de dados **mais simples** é a **pilha**.

- A **ideia fundamental** da estrutura é que **todo o acesso** aos dados seja feito a partir de uma **determinada posição**, ou **topo**.

Introdução

- Quando um elemento novo é **inserido** ou **empilhado**, ele passa a ser o **elemento do topo**.

- O **acesso imediato** acontece **somente** com o elemento do topo.

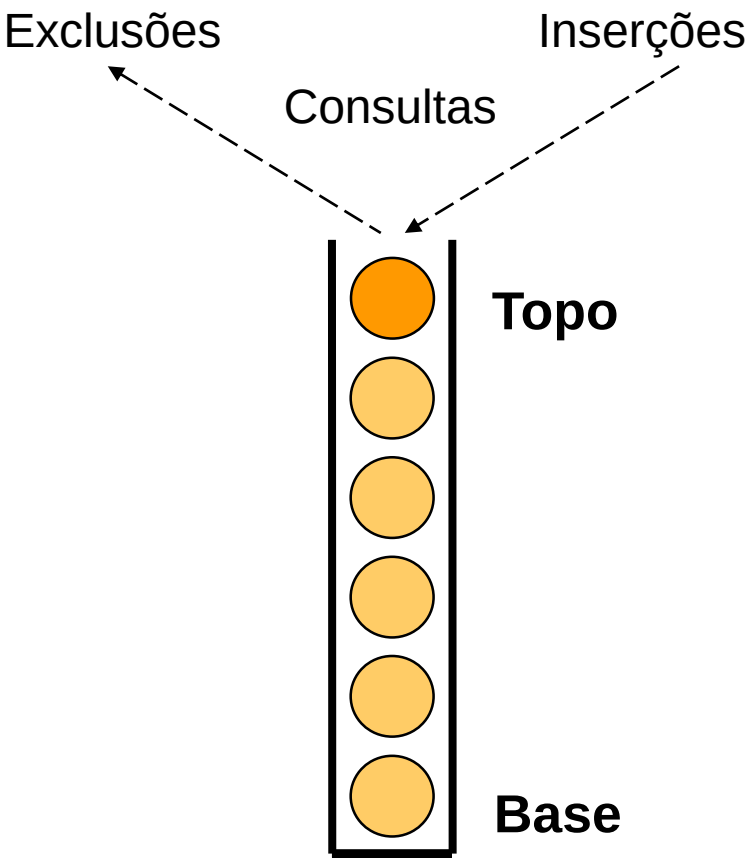
- O **primeiro elemento** que pode ser **removido** é o elemento do topo.

Introdução

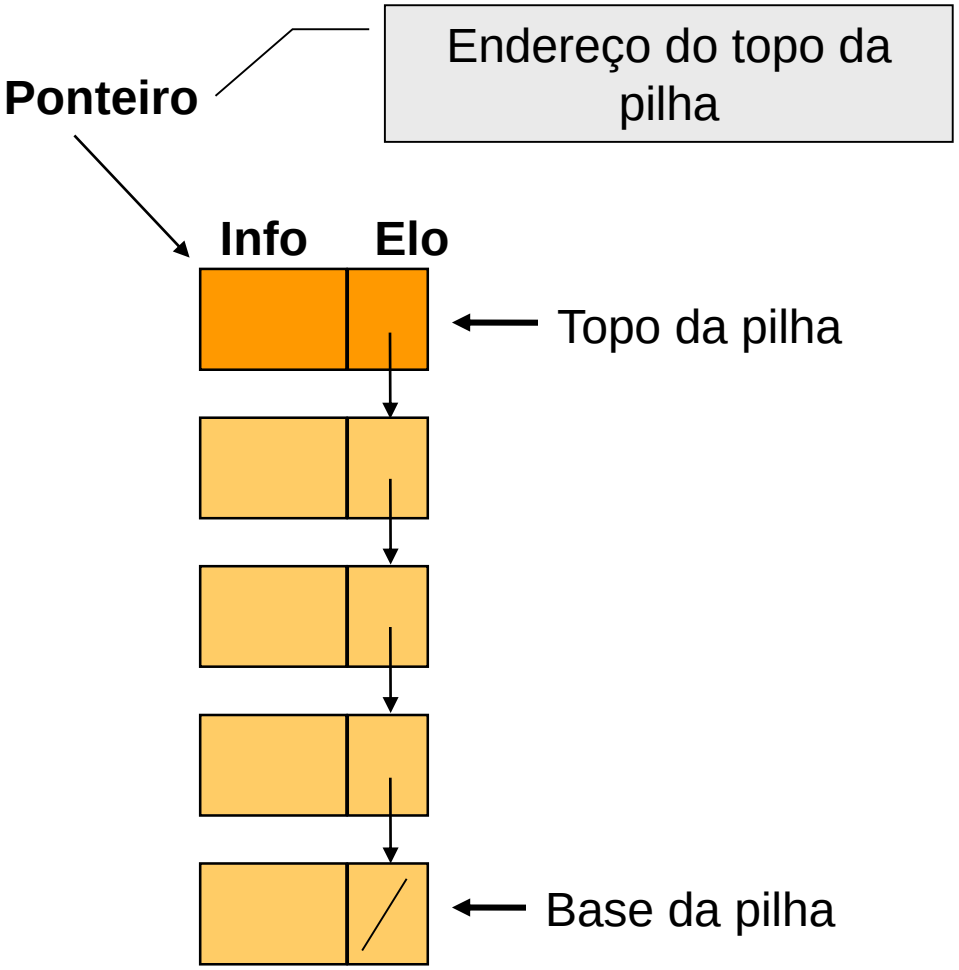
- Com isso, os elementos da pilha só podem ser retirados na **ordem inversa** à ordem em que foram introduzidos.

- O primeiro elemento removido é o último elemento inserido. Esse conceito é representado pela sigla LIFO (***Last In***, ***First Out***).

Representação



Representação



Operações
Pilha

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a pilha
  - **verificar** se a pilha está **vazia**
  - **empilhar item** na pilha
  - **desempilhar item** da pilha
  - **verificar** o **tamanho** da pilha

Implementação

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef int TipoChave;

typedef struct{
  int chave;
  /*outros componentes podem ser adicionados*/
} TipoItem;
```

Implementação

```
typedef struct TipoCelula *TipoApontador;
typedef struct TipoCelula {
  TipoItem item;
  TipoApontador proximo;
} TipoCelula;
```

```
typedef struct {
  int tamanho;
  TipoApontador fundo, topo;
} TipoPilha;
```

Implementação

```
void Inicializa(TipoPilha *pilha){
  pilha->topo =
     (TipoApontador) malloc (sizeof(TipoCelula));
  pilha->fundo = pilha->topo;
  pilha->topo->proximo = NULL;
  pilha->tamanho = 0;
}
```

```
int Vazia(TipoPilha pilha){
  return (pilha.topo == pilha.fundo);
}
```

Implementação

```
void Empilha(TipoItem elemento,
                                TipoPilha *pilha){
  TipoApontador p_auxiliar;
  p_auxiliar =
     (TipoApontador) malloc (sizeof(TipoCelula));
  pilha->topo->item = elemento;
  p_auxiliar->proximo = pilha->topo;
  pilha->topo = p_auxiliar;
  pilha->tamanho++;
}
```

Implementação

```
void Desempilha(TipoPilha *pilha,
                                        TipoItem *item){
  TipoApontador p_auxiliar;
  if (Vazia(*pilha)){
    printf("Erro: lista vazia\n");
    return;
  }
  p_auxiliar = pilha->topo;
  pilha->topo = p_auxiliar->proximo;
  *item = p_auxiliar->proximo->item;
  free(p_auxiliar);
  pilha->tamanho--;
}
```

Implementação

```
int Tamanho(TipoPilha pilha){
  return (pilha.tamanho);
}
```

Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Verificar** o **primeiro elemento** da pilha
  - **Contar** o **número de itens** da pilha
  - **Imprimir** elementos da pilha

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Aulas Professor Rodolfo Miranda Pereira, UEM.